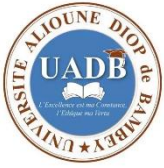


Université Alioune Diop (UAD), Bambey



Sciences Appliquées et Technologies de l'Information et de la
Communication



Département : Technologies de l'Information et de la Communication

Master 1 : Système d'Information

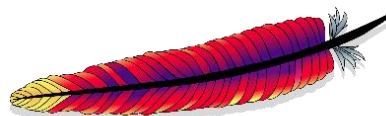
Travaux pratiques

Utiliser Ansible pour automatiser l'installation

D'un serveur Web



ANSIBLE



Apache Web Server

Présenté par :

Gana Faye

Table des matières

Partie 1 : Lancer la machine virtuelle DEVASC	3
Partie 2 : Configurer Ansible.....	4
Partie 3 : Vérifier les communications avec le serveur Web local	7
Partie 4 : Créer des Playbooks Ansible pour automatiser l'installation du serveur Web	9
Partie 5 : Ajouter des options à votre Playbook Ansible pour serveurs Web Apache	12
Conclusion du Rapport	15

Partie 1 : Lancer la machine virtuelle DEVASC

Le déploiement et le test de solutions d'automatisation nécessitent un environnement isolé et contrôlé. Pour ce TP, on a utilisé la machine virtuelle **DEVASC-LABVM**, qui est une station de travail Ubuntu spécifiquement configurée pour le développement et l'automatisation réseau (DevNet).

Mise en œuvre

1. **Lancement de l'hyperviseur** : Utilisation de VirtualBox pour démarrer l'instance DEVASC-LABVM.
2. **Initialisation du système** : Une fois la VM démarrée, j'ai accédé à l'interface utilisateur avec l'identifiant par défaut devasc.
3. **Vérification de l'environnement** : J'ai validé que les outils nécessaires (Python, Ansible, VS Code) étaient présents et opérationnels pour débiter la configuration.

Caractéristiques de la VM

Cette machine virtuelle simule un environnement de production où les interfaces réseaux (comme dummy0) permettent de tester des déploiements de serveurs web sans impacter le réseau physique de l'hôte.

Partie 2 : Configurer Ansible

L'objectif de cette étape est de préparer l'environnement de contrôle pour qu'Ansible puisse identifier et communiquer avec le serveur cible.

Bien que le serveur soit simulé localement sur l'interface dummy0 (IP 192.0.2.3), Ansible le traite comme une machine distante via le protocole SSH.

Étape 1 & 2 : Préparation du service SSH

Le protocole SSH est le "moteur" de transport d'Ansible. Par défaut, le service était désactivé sur la VM DEVASC.

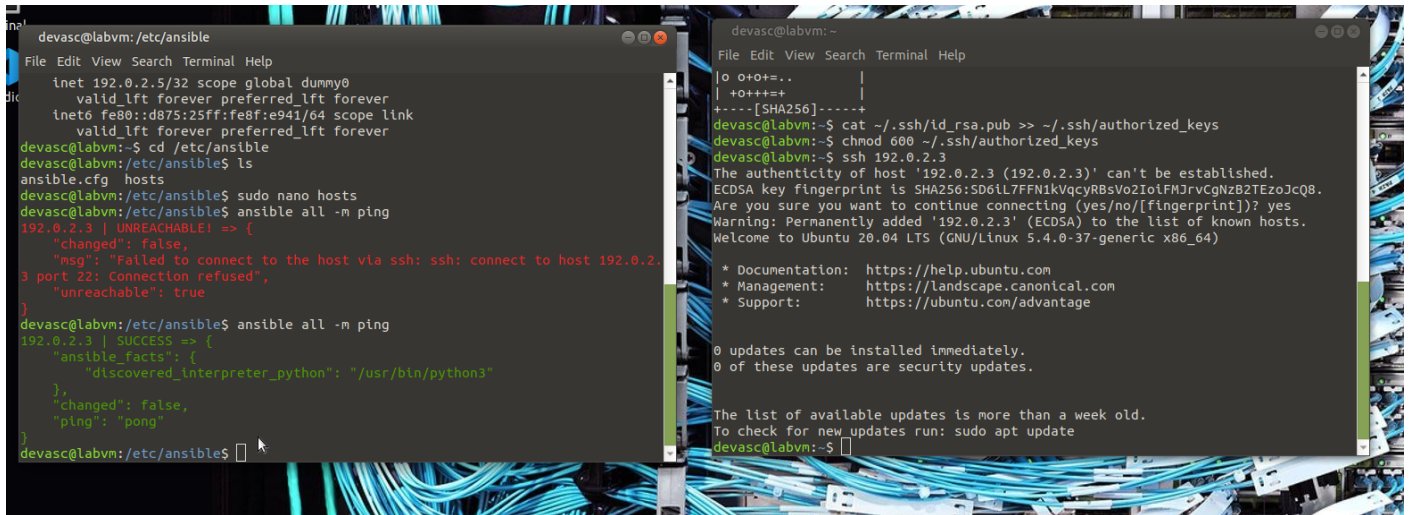
- **Action** : Activation du serveur SSH pour permettre les connexions entrantes.
- Utilisation de sshpass pour permettre à Ansible de transmettre le mot de passe de manière automatisée lors de la phase de test (en production, on privilégiera les clés SSH).

```
devasc@labvm:~$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/devasc/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/devasc/.ssh/id_rsa
Your public key has been saved in /home/devasc/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:8BexptqyRkSgBxqn3dry8F97Kna9F36alnId0MigAQ devasc@labvm
The key's randomart image is:
+--[RSA 3072]-----+
|E.          |
|o          .o|
|o          .+|
| = .o  =    |
|o * =.o 5 . |
|.o00000 .   |
|o *o0++    |
|o o+o++..   |
|+0+++++    |
+-----[SHA256]-----+
devasc@labvm:~$ cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
devasc@labvm:~$ chmod 600 ~/.ssh/authorized_keys
devasc@labvm:~$ ssh 192.0.2.3
The authenticity of host '192.0.2.3 (192.0.2.3)' can't be established.
ECDSA key fingerprint is SHA256:5D6LL7FFNkKvqcyRBSvoZtaFPJ3vcghB2IEzoJcQ8.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.0.2.3' (ECDSA) to the list of known hosts.
Welcome to Ubuntu 20.04 LTS (GNU/Linux 5.4.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

0 updates can be installed immediately.
0 of these updates are security updates.

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
devasc@labvm:~$
```



The image shows two terminal windows from a lab VM. The left window shows the user navigating to `/etc/ansible`, listing files, editing `hosts` with `nano`, and running `ansible all -m ping`. The output shows a connection failure for `192.0.2.3` on port 22. The right window shows the user adding a public key to `authorized_keys`, running `ssh 192.0.2.3`, and accepting the host's fingerprint. It then shows the system's update status, indicating that updates are available.

```
devasc@labvm: /etc/ansible
File Edit View Search Terminal Help
inet 192.0.2.5/32 scope global dummy0
valid_lft forever preferred_lft forever
inet6 fe80::d875:25ff:fe8f:e941/64 scope link
valid_lft forever preferred_lft forever
devasc@labvm:~$ cd /etc/ansible
devasc@labvm:/etc/ansible$ ls
ansible.cfg  hosts
devasc@labvm:/etc/ansible$ sudo nano hosts
devasc@labvm:/etc/ansible$ ansible all -m ping
192.0.2.3 | UNREACHABLE! => {
  "changed": false,
  "msg": "Failed to connect to the host via ssh: ssh: connect to host 192.0.2.3 port 22: Connection refused",
  "unreachable": true
}
devasc@labvm:/etc/ansible$ ansible all -m ping
192.0.2.3 | SUCCESS ==> {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
devasc@labvm:/etc/ansible$
```

```
devasc@labvm:~$ cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
devasc@labvm:~$ chmod 600 ~/.ssh/authorized_keys
devasc@labvm:~$ ssh 192.0.2.3
The authenticity of host '192.0.2.3 (192.0.2.3)' can't be established.
ECDSA key fingerprint is SHA256:SD6iL7FFN1kVqcyRBsVo2IoIFMJrvCgNzB2TEzoJcQ8.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.0.2.3' (ECDSA) to the list of known hosts.
Welcome to Ubuntu 20.04 LTS (GNU/Linux 5.4.0-37-generic x86_64)

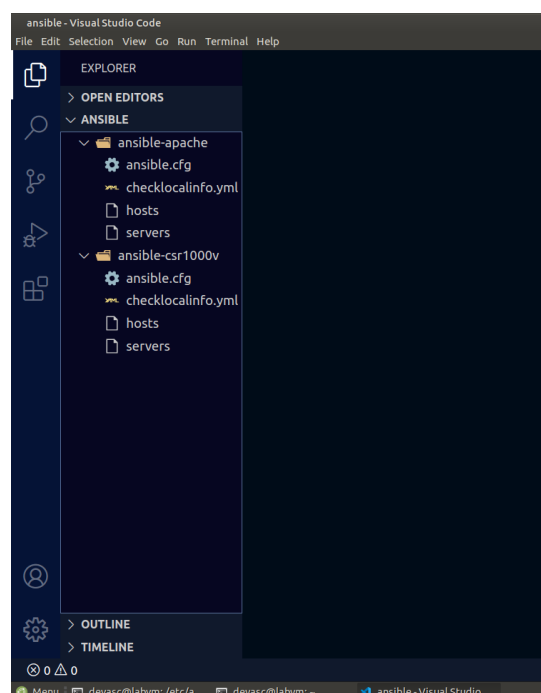
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

0 updates can be installed immediately.
0 of these updates are security updates.

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
devasc@labvm:~$
```

Étape 3 : Environnement de développement (VS Code)

Pour une gestion efficace des scripts, le dossier `~/labs/devnet-src/ansible/ansible-apache` a été ouvert dans Visual Studio Code. Cette approche permet de visualiser l'arborescence du projet et de modifier simultanément l'inventaire et les playbooks.



Étape 4 : Définition de l'inventaire (hosts)

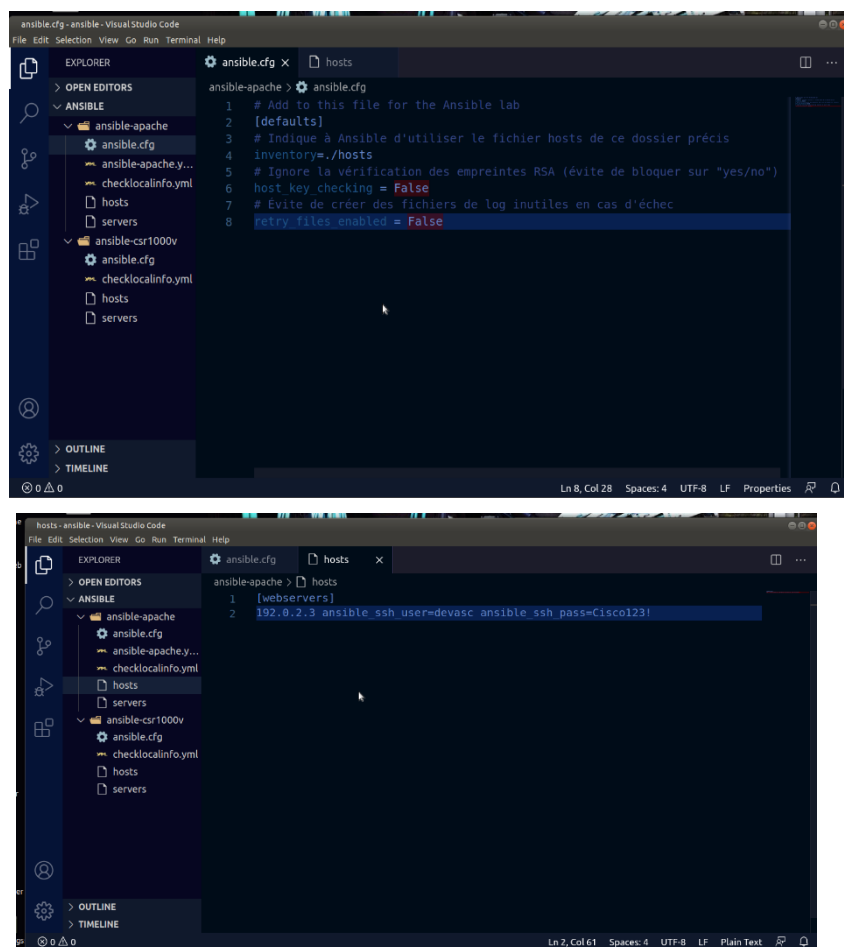
J'ai configuré le groupe [webservers] pour inclure notre serveur cible :

- **Configuration** : 192.0.2.3 ansible_ssh_user=devasc ansible_ssh_pass=Cisco123!
- **Analyse** : Cette ligne définit non seulement l'adresse IP, mais aussi les paramètres de connexion nécessaires pour qu'Ansible puisse s'authentifier sans intervention humaine.

Étape 5 : Personnalisation du comportement d'Ansible (ansible.cfg)

Le fichier ansible.cfg permet de centraliser les paramètres de fonctionnement. Voici les directives configurées :

- `inventory=./hosts` : Force Ansible à utiliser le fichier d'inventaire local plutôt que le fichier système `/etc/ansible/hosts`.
- `host_key_checking = False` : Désactive la vérification des empreintes RSA (utile en laboratoire pour éviter les blocages lors de la première connexion).
- `retry_files_enabled = False` : Empêche la création de fichiers `.retry` en cas d'échec, gardant le dossier de travail propre.



Une vue de VS Code montrant les deux fichiers (hosts et ansible.cfg)

Partie 3 : Vérifier les communications avec le serveur Web local

Avant de procéder à l'automatisation complexe, il est impératif de valider la connectivité entre le nœud de contrôle Ansible et les hôtes cibles. Cette étape confirme que la configuration SSH et l'inventaire réalisés en Partie 2 sont fonctionnels.

Étape 1 : Validation de la connectivité via le module ping

La première vérification s'effectue avec le module ping. Contrairement à la commande ICMP classique, le module ping d'Ansible vérifie non seulement que l'hôte est joignable, mais aussi qu'une session Python peut être initialisée via SSH.

- **Commande exécutée :** `ansible webservers -m ping` ou **`ansible all -m ping`** qui fait la même chose
- **Analyse du résultat :** La réponse "ping": "pong" confirme que les informations d'authentification (*ansible_ssh_user* et *ansible_ssh_pass*) fournies dans le fichier hosts sont correctes et que l'environnement est prêt.



```
netto world
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$ ansible all -m ping
192.0.2.3 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$
```

Étape 2 : Test de communication via le module command

Pour aller plus loin, j'ai testé la capacité d'Ansible à exécuter des instructions directement sur le système distant en utilisant le module command.

- **Commande exécutée :** `ansible webservers -m command -a "/bin/echo hello world"`
- **Détails techniques :** L'argument -a permet de passer la chaîne de caractères à exécuter.
 - Le serveur a répondu avec succès par hello world.
- **Observation :** Le terminal affiche le statut CHANGED. Cela indique qu'Ansible a bien initié une tâche sur la cible.

The screenshot shows the Visual Studio Code interface with the 'ansible.cfg' file open in the editor. The file contains the following content:

```
1 # Add to this file for the Ansible lab
2 [defaults]
3 # Indique à Ansible d'utiliser le fichier hosts de ce dossier précis
4 inventory=./hosts
5 # Ignore la vérification des empreintes RSA (évite de bloquer sur "yes/no")
6 host_key_checking = False
7 # Évite de créer des fichiers de log inutiles en cas d'échec
8 retry_files_enabled = False
```

The terminal window at the bottom shows the execution of two Ansible commands:

```
devasc@labvm:~/Labs/devnet-src/ansible/ansible-apache$ ansible webservers -m command -a "/bin/echo hello world"
192.0.2.3 | CHANGED | rc=0 >>
hello world
devasc@labvm:~/Labs/devnet-src/ansible/ansible-apache$ ansible all -m ping
192.0.2.3 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

The terminal output for both commands shows 'SUCCESS' in green text, indicating successful execution.

Une capture de ton terminal montrant les deux commandes (ping et echo) avec les blocs de texte en **jaune** ou **vert** confirmant le succès ("SUCCESS").

Partie 4 : Créer des Playbooks Ansible pour automatiser l'installation du serveur Web

L'utilisation des Playbooks permet de passer à l'échelle (*scaling*).

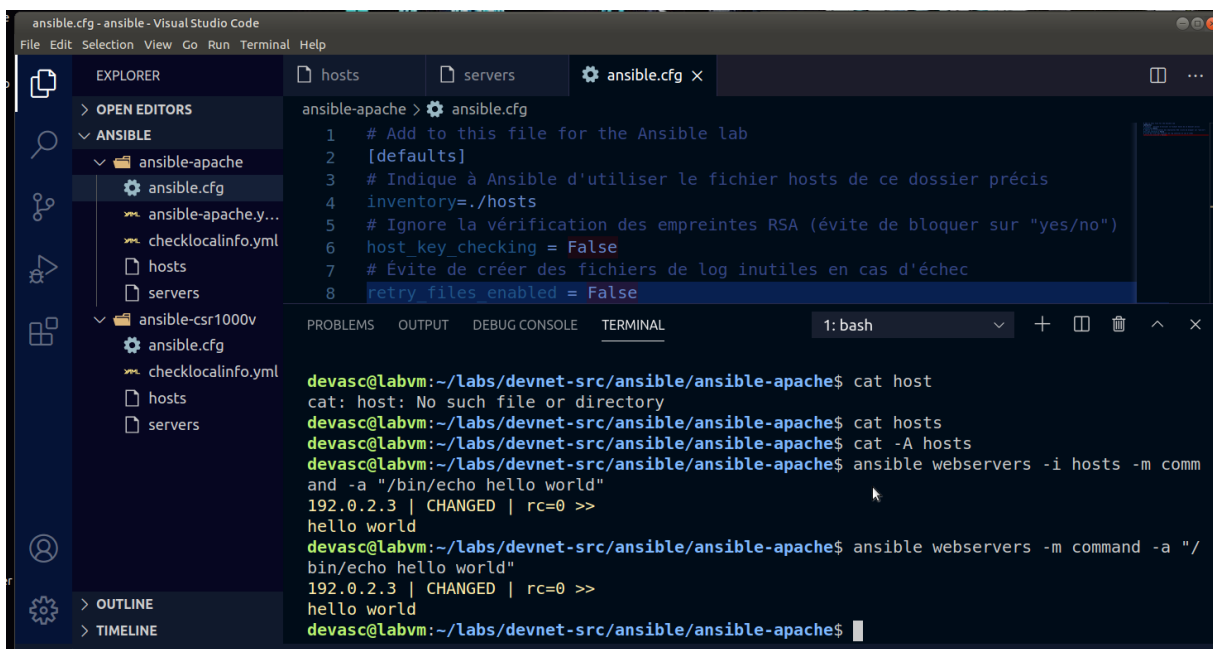
Au lieu d'exécuter des commandes une par une, nous définissons un état souhaité dans des fichiers YAML.

Ansible se charge ensuite d'appliquer cette configuration à l'ensemble du groupe webservers.

Étape 1 & 2 : Automatisation du test de communication

D'abord créé un playbook simple, `test_apache_playbook.yaml`, pour automatiser la vérification de la connectivité.

- **Structure du fichier :** Ce playbook définit une tâche unique utilisant le module `command` pour exécuter `/bin/echo hello world`.
- **Exécution :** La commande utilisée est `ansible-playbook -v test_apache_playbook.yaml`.
- **Résultat :** L'option `-v` (verbose) a permis de voir le détail du retour de la commande (stdout: "hello world") et de confirmer que le moteur Ansible fonctionne correctement sur la cible.



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project structure with folders `ansible-apache` and `ansible-csr1000v`, each containing `ansible.cfg`, `checklocalinfo.yml`, `hosts`, and `servers`. The main editor displays the content of `ansible.cfg` for the `ansible-apache` folder. The file contains the following configuration:

```
1 # Add to this file for the Ansible lab
2 [defaults]
3 # Indique à Ansible d'utiliser le fichier hosts de ce dossier précis
4 inventory=./hosts
5 # Ignore la vérification des empreintes RSA (évite de bloquer sur "yes/no")
6 host_key_checking = False
7 # Évite de créer des fichiers de log inutiles en cas d'échec
8 retry_files_enabled = False
```

The TERMINAL pane at the bottom shows a series of commands and their outputs:

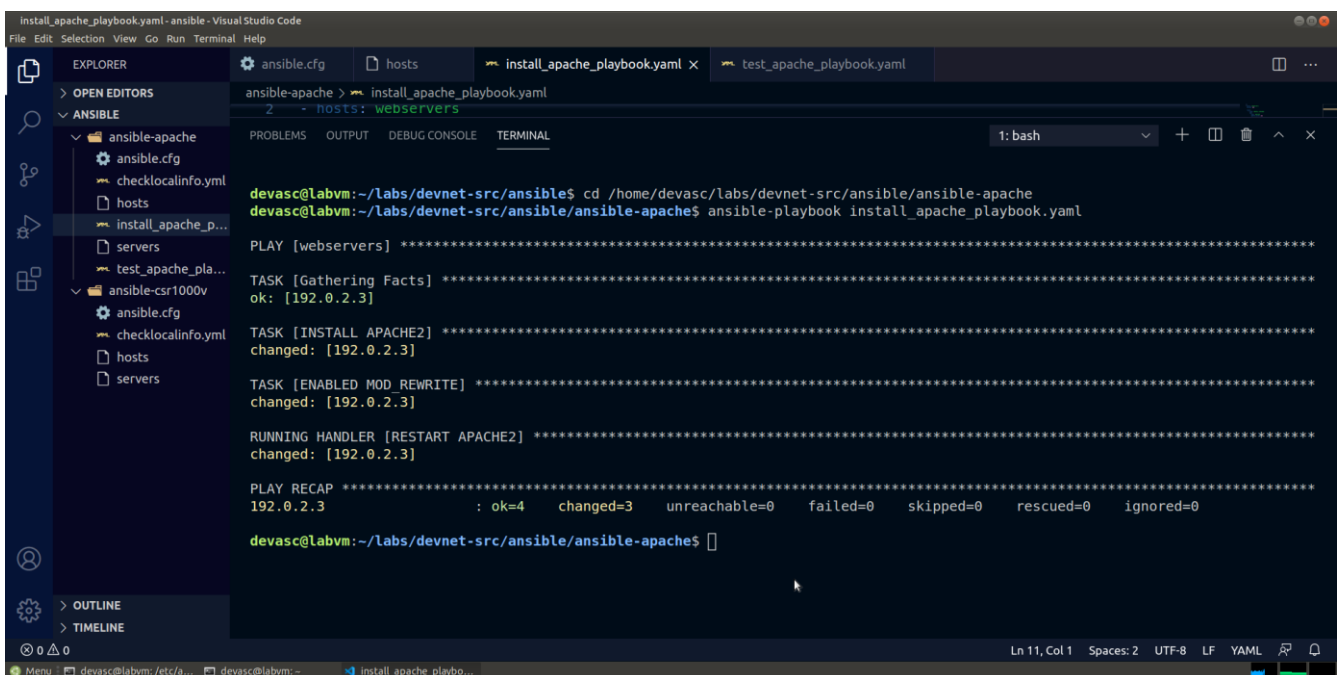
```
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$ cat host
cat: host: No such file or directory
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$ cat hosts
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$ cat -A hosts
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$ ansible webservers -i hosts -m comm
and -a "/bin/echo hello world"
192.0.2.3 | CHANGED | rc=0 >>
hello world
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$ ansible webservers -m command -a "/
bin/echo hello world"
192.0.2.3 | CHANGED | rc=0 >>
hello world
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$
```

Étape 3 & 4 : Création du Playbook d'installation d'Apache

L'étape suivante a consisté à créer `install_apache_playbook.yaml`, un script beaucoup plus complexe gérant l'installation et la configuration du service.

Analyse des composants clés :

- **become: yes** : Indispensable pour obtenir les privilèges `sudo` nécessaires à l'installation de paquets.
- **Module apt** : Utilisé pour installer `apache2`. Le paramètre `state: latest` garantit que nous disposons de la version la plus récente, tandis que `update_cache: yes` synchronise les dépôts avant l'installation.
- **Module apache2_module** : Permet d'activer le module `rewrite`.
- **handlers & notify** : C'est une fonctionnalité avancée. Le redémarrage du service (RESTART APACHE2) n'est déclenché que si la tâche d'activation du module `rewrite` signale un changement (`changed`). Cela optimise la disponibilité du serveur.



```
devasc@labvm:~/labs/devnet-src/ansible$ cd /home/devasc/labs/devnet-src/ansible/ansible-apache
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$ ansible-playbook install_apache_playbook.yaml

PLAY [webservers] *****

TASK [Gathering Facts] *****
ok: [192.0.2.3]

TASK [INSTALL APACHE2] *****
changed: [192.0.2.3]

TASK [ENABLED MOD_REWRITE] *****
changed: [192.0.2.3]

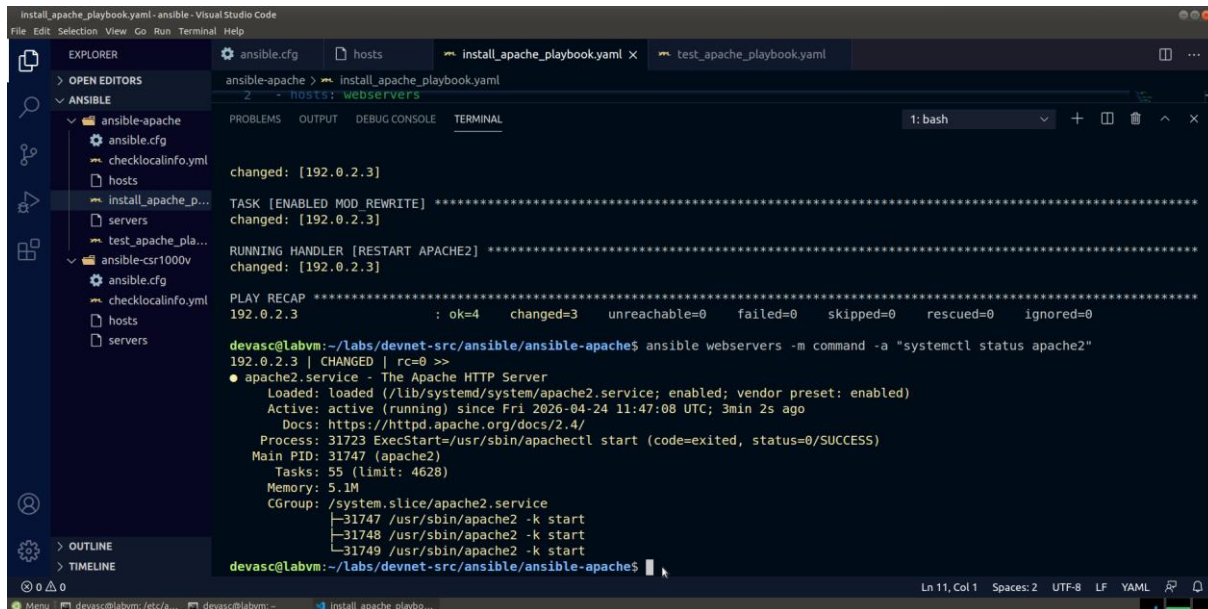
RUNNING HANDLER [RESTART APACHE2] *****
changed: [192.0.2.3]

PLAY RECAP *****
192.0.2.3 : ok=4  changed=3  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$
```

Étape 5 : Exécution du déploiement

Le lancement du playbook a permis d'observer la gestion de l'**idempotence**. Lors de la première exécution, Ansible a installé les composants (statut changed). Si l'on relançait le script, Ansible détecterait que tout est déjà en place et n'effectuerait aucune modification (statut ok).



```
install_apache_playbook.yaml - ansible - Visual Studio Code
File Edit Selection View Go Run Terminal Help

EXPLORER
  ansible.cfg
  hosts
  install_apache_playbook.yaml
  test_apache_playbook.yaml

ANSIBLE
  ansible-apache
    ansible.cfg
    checklocalinfo.yml
    hosts
    install_apache_p...
    servers
  test_apache_pla...
    ansible.cfg
    checklocalinfo.yml
    hosts
    servers

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
1: bash

changed: [192.0.2.3]
TASK [ENABLED MOD_REWRITE] *****
changed: [192.0.2.3]
RUNNING HANDLER [RESTART APACHE2] *****
PLAY RECAP *****
192.0.2.3 : ok=4 changed=3 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0

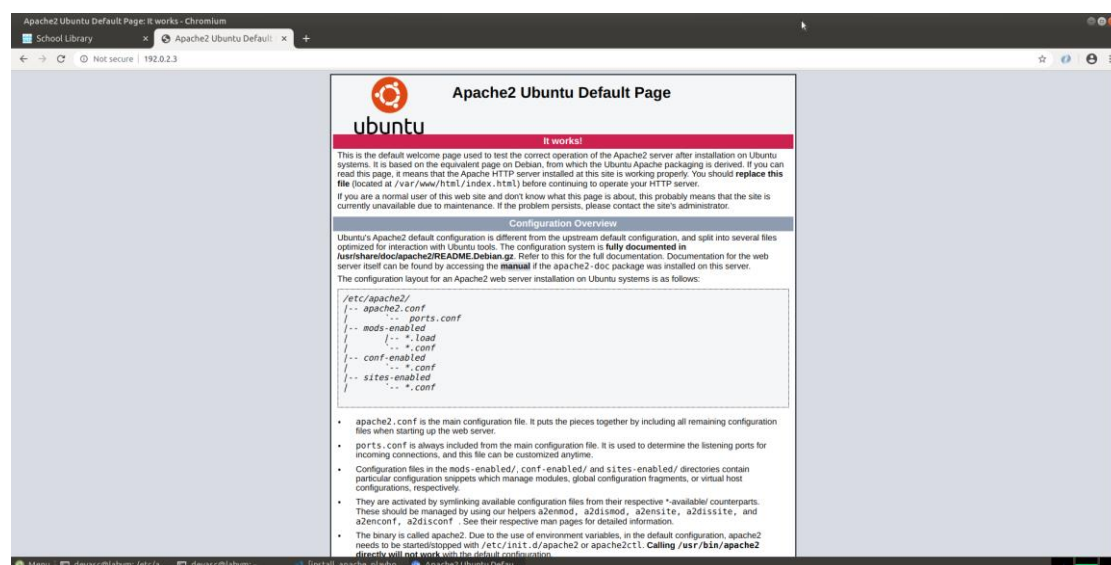
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$ ansible webserver -m command -a "systemctl status apache2"
192.0.2.3 | CHANGED | rc=0 >>
  ● apache2.service - The Apache HTTP Server
    Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
    Active: active (running) since Fri 2026-04-24 11:47:08 UTC; 3min 2s ago
    Docs: https://httpd.apache.org/docs/2.4/
    Process: 31723 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
    Main PID: 31747 (apache2)
    Tasks: 55 (limit: 4628)
    Memory: 5.1M
    CGroup: /system.slice/apache2.service
            └─31747 /usr/sbin/apache2 -k start
            └─31748 /usr/sbin/apache2 -k start
            └─31749 /usr/sbin/apache2 -k start

devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$
```

Étape 6 : Vérification de la réussite

La validation finale a été effectuée par deux méthodes :

1. **Ligne de commande** : Vérification du statut du service avec `systemctl status apache2`.
2. **Interface graphique** : Navigation sur `http://192.0.2.3` via Chromium, affichant la page par défaut "Apache2 Ubuntu Default Page".



Partie 5 : Ajouter des options à votre Playbook Ansible pour serveurs Web Apache

L'objectif de cette partie est d'utiliser Ansible pour modifier la configuration réseau d'Apache, en changeant son port d'écoute par défaut (80) vers un port personnalisé (8081). Cela démontre la puissance d'Ansible pour manipuler des fichiers de configuration sans intervention manuelle.

Étape 1 : Création du Playbook avec options

J'ai créé le fichier `install_apache_options_playbook.yaml`. Ce playbook introduit le module **lineinfile**, qui est essentiel pour la modification chirurgicale de fichiers textes.

- **Module lineinfile** : Il permet de rechercher une ligne spécifique via une expression régulière (regex) et de la remplacer par une nouvelle ligne (line).
- **Cibles de modification** :
 1. `/etc/apache2/ports.conf` : Pour changer le port d'écoute global du service.
 2. `/etc/apache2/sites-available/000-default.conf` : Pour mettre à jour la directive du VirtualHost afin qu'elle corresponde au nouveau port.

Étape 2 : État initial du système

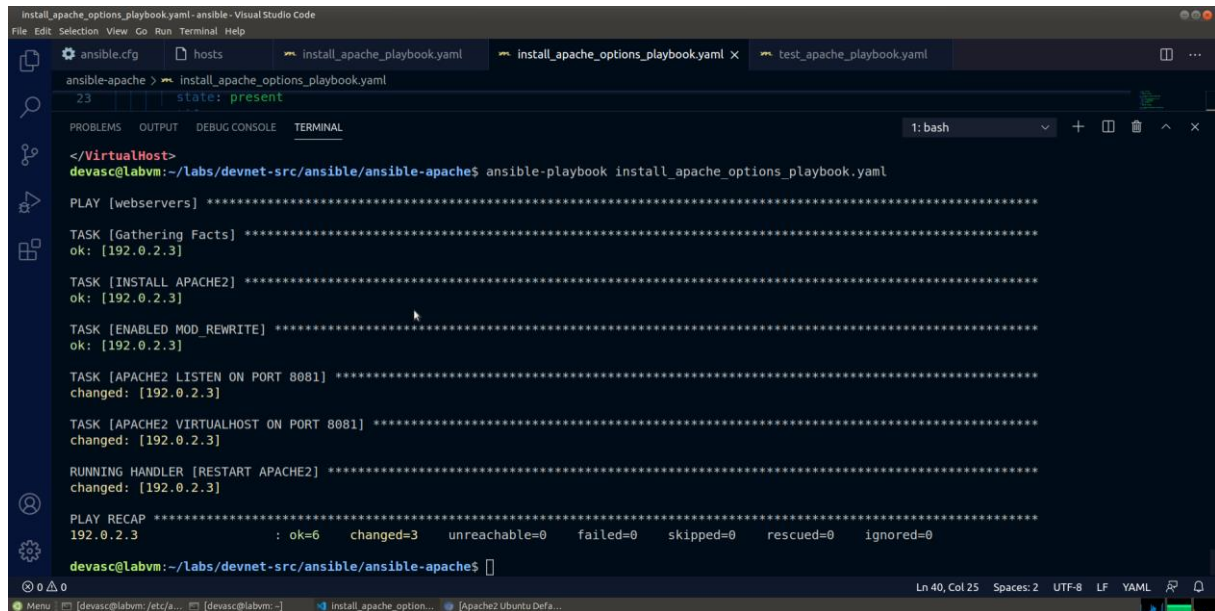
Avant l'exécution, j'ai vérifié les fichiers de configuration manuellement. Les directives indiquaient `Listen 80` et `<VirtualHost *:80>`, confirmant que le serveur pointait sur le port HTTP standard.



```
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$ grep "Listen" /etc/apache2/ports.conf
Listen 80
      Listen 443
      Listen 443
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$ grep "VirtualHost" /etc/apache2/sites-available/000-default.conf
<VirtualHost *:80>
</VirtualHost>
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$
```

Étape 3 : Exécution et automatisation du changement

L'exécution du playbook a déclenché les changements. Grâce au système de **Handlers** (notify: RESTART APACHE2), Ansible a automatiquement redémarré Apache dès qu'il a détecté la modification des fichiers .conf. Sans ce redémarrage automatisé, le serveur aurait continué à écouter sur le port 80 malgré les changements de fichiers.



```
install_apache_options_playbook.yml - ansible - Visual Studio Code
File Edit Selection View Go Run Terminal Help

ansible.cfg  hosts  install_apache_playbook.yml  install_apache_options_playbook.yml x  test_apache_playbook.yml

ansible-apache > install_apache_options_playbook.yml
23 | state: present

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL 1: bash

</VirtualHost>
devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$ ansible-playbook install_apache_options_playbook.yml

PLAY [webservers] *****

TASK [Gathering Facts] *****
ok: [192.0.2.3]

TASK [INSTALL APACHE2] *****
ok: [192.0.2.3]

TASK [ENABLED MOD_REWRITE] *****
ok: [192.0.2.3]

TASK [APACHE2 LISTEN ON PORT 8081] *****
changed: [192.0.2.3]

TASK [APACHE2 VIRTUALHOST ON PORT 8081] *****
changed: [192.0.2.3]

RUNNING HANDLER [RESTART APACHE2] *****
changed: [192.0.2.3]

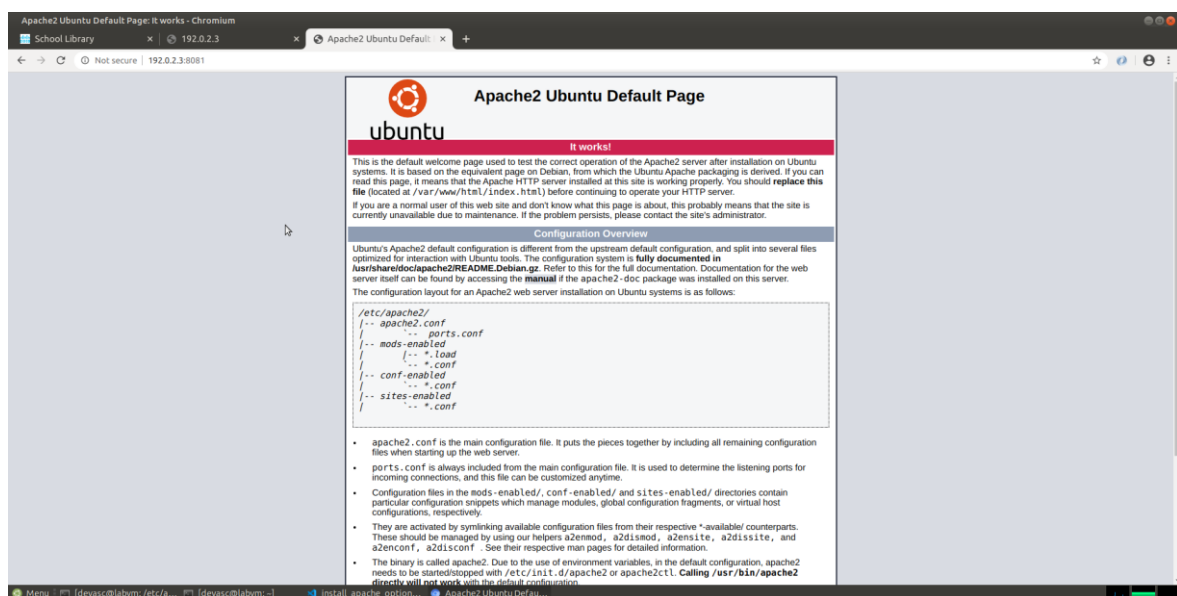
PLAY RECAP *****
192.0.2.3 : ok=6 changed=3 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0

devasc@labvm:~/labs/devnet-src/ansible/ansible-apache$
```

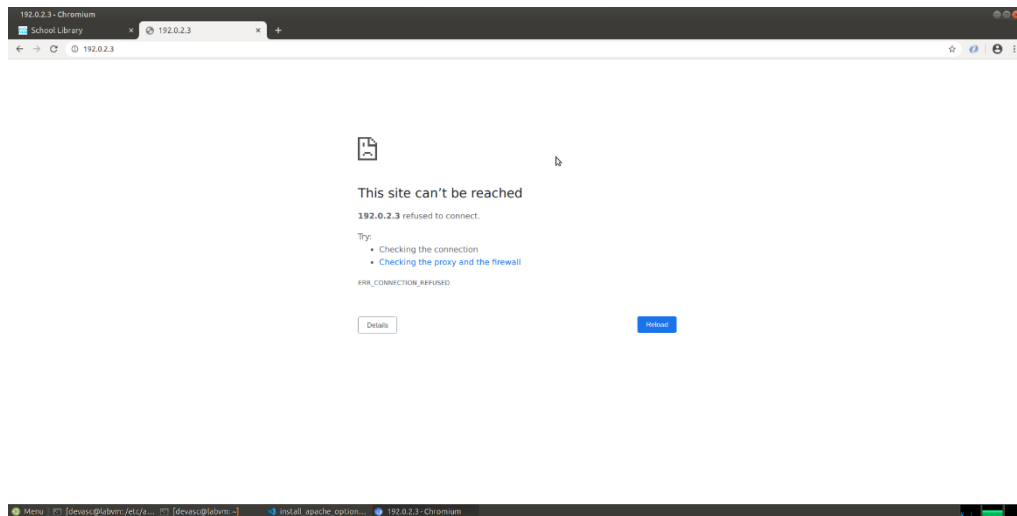
Étape 4 : Vérification et validation finale

La réussite de cette configuration a été confirmée par deux tests :

1. **Validation interne** : Lecture des fichiers /etc/apache2/ports.conf montrant désormais Listen 8081.



2. **Test de service :** Tentative de connexion via Chromium. L'accès à 192.0.2.3 (port 80) échoue désormais, tandis que l'accès à 192.0.2.3:8081 affiche correctement la page d'accueil d'Apache.



Conclusion du Rapport

Ce travail pratique a permis de démontrer l'efficacité d'Ansible comme outil d'**Infrastructure as Code (IaC)**.

- **Fiabilité** : L'utilisation de modules comme apt et lineinfile réduit les erreurs de saisie humaine.
- **Rapidité** : Ce qui prendrait plusieurs minutes manuellement (installation, activation de modules, modification de fichiers, redémarrage) est exécuté en quelques secondes.
- **Sécurité** : En tant que futur spécialiste en cybersécurité, je retiens qu'Ansible permet de déployer des configurations durcies et identiques sur l'ensemble d'un parc, garantissant ainsi qu'aucun serveur ne soit oublié lors des mises à jour de sécurité.